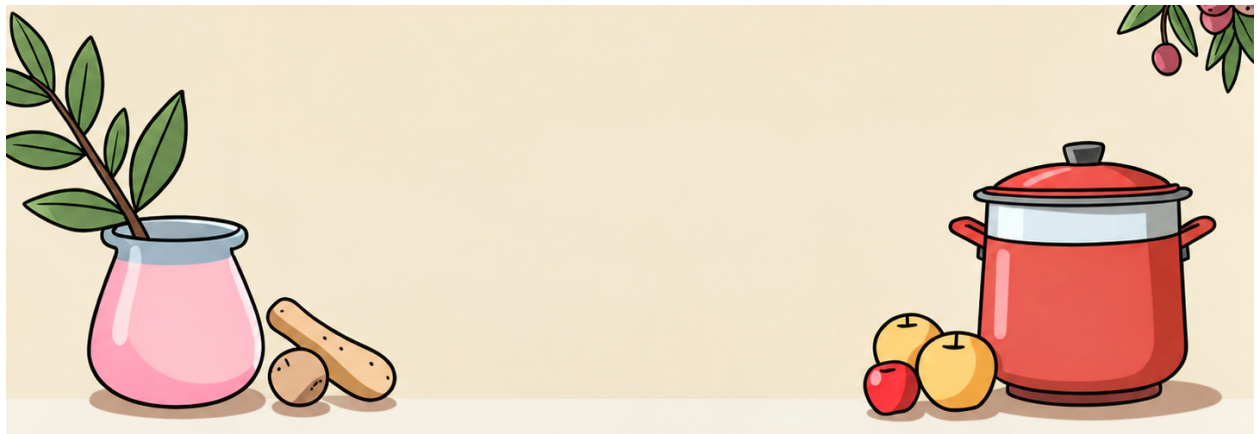




Farm-to-Table: Building an iOS App with Local LLMs

Lessons from the test kitchen

I posted a little while ago about an experiment I've been working on for the past few weeks. I was really surprised at the reactions and questions I got, so I thought it would be worth taking a pause (while my coding agent crunches away) to share where I'm at and give anyone reading a look into what I've got cooking.

More than 10 years have passed since I dreamed up the idea for an app where I could store all the recipes me and my wife have accumulated over the years. I know there are probably tons of recipe apps available, but none of them are... mine. By and large, I don't always get much time to spend coding on my own time. Between work, family, and all the activities life throws at us, it's really hard to sit down and write a lot of code. Usually when I get inspired, or find a really fun side-project is when I disappear into my MacBook. But of course LLMs have changed the game recently. So I figured, with an itch to do some development that needed scratching, I could finally dive in.

The only problem - I can be kinda cheap when it comes to paying for a service when I'm not really sure I need it. I've hemmed and hawed over other such purchases, so when it came to locking into Claude, Cursor, ChatGPT, or something else - I just couldn't get myself to pull the trigger. Call me a cheapskate.

With that in mind, I had some inspiration - could my MacBook Pro from 2021 actually run some basic models locally, and would they be good enough to actually do agentic coding to build a real app? I'm not in a rush to get the app published just yet, so I decided to take on the challenge. Full disclaimer - For the earlier ideation, basic structure, and setup, I did use Claude (free version) to get myself set up. Probably could skip that, but I don't consider it breaking the rules. ^_^

Although my app isn't quite done, I think I'm far enough along in the process to share some tips in case you're interested in trying any of this yourself.

Before we go any further, here's the TL;DR of my setup:

- Hardware: Macbook Pro 2021 M1 Max, 64GB RAM
- LLM Setup
 - VS Code IDE

- Cline
- Ollama running the models
- Model: `qwen3.6:35b-a3b-q8_0`

How did I get here, and if you're brave enough to follow me, what tips can I share?

1. *Write down your plan - and then some*
 2. *Keep your work bite sized*
 3. *Get to good and go*
 4. *Know when (and be ready) to step in*
 5. *I say patience*
-



1. Write down your plan - and then some

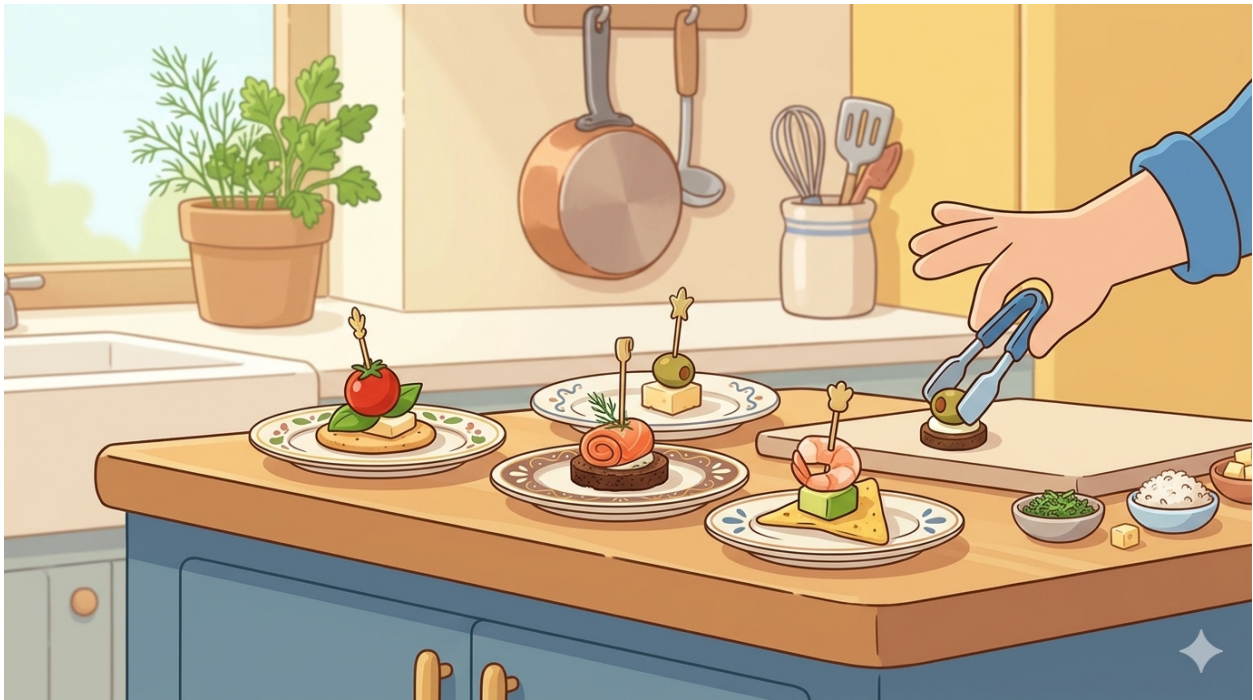
This is the step where you might argue I cheated, because I didn't actually use local LLMs at all. This was more about convenience and speed, more than anything else, but I never actually paid for a subscription, so I'm calling this one a win regardless.

Local models, no matter how good your setup, aren't going to carry the same massively remembered context a large hosted model does. So if you want any success, it really pays to have a good plan up front when starting any project or task. This is where I'd spend the first portion of your time and energy. Can you get a local model to do this for you too - absolutely! Indeed, since it's just a lot of text at this point, you could easily use a chat interface and a local model to knock out a lot of this.

Taking this a bit further - you should probably assume that for each story or milestone you work on, you make this your first step. If you ask your LLM to produce a really detailed coding plan, you're far more likely to have success when you ask the agent to actually write the code.

Some good outputs to consider - detailed requirements, an architecture diagram, decision logs, and an explicit task list are usually things I have in place before any actual coding happens.

Example prompt: We are working on the next feature in the app. You will propose a detailed implementation plan to work on this, and write a md file at the root of this repo with the plan that an agent can work through.



2. Keep your work bite sized

Local models work best when they have a defined task to work through. A single task or story almost always works better than having your Agent try to rip through multiple aspects at once. For example, don't ask it to build the Models, Services and UI all at once. It might get it done, but has a much higher chance of just timing out, hallucinating code, or just spinning its wheels for a really long time. None of this is the outcome you're looking for.

Building on the first point, before working on a feature or task, explicitly tell your model to build the task list (if you didn't already). Then ask the agent to work on only one part of the plan, and that's it.

Example prompt: **You are a coding agent who will work on task 5 in the feature-tasklist.md. Do not do any other tasks. Just read the plan, and make all appropriate code changes. When you are done, please pause, and i will build and run, and run any tests, to determine our next step.**

As you can see, we're being very clear to take on only a single task (which usually comprises no more than a few classes and associated tests).

This can feel a bit slow and frustrating (see points below), but your results are far more likely to be higher quality with a lot fewer failures and wasted energy.



3. Get to good and go

Before this feels like it's actually viable, assume you're going to spend a good amount of time playing with different harnesses, interaction approaches, LLMs, etc. Additionally, you'll probably spend a decent amount of time getting your agent guidelines tweaked as well. Trust me, this is probably worth it, because the setup can often be confusing and riddled with a seemingly infinite set of options.

If you're on a Mac, you have a ton of ways to run models. Ollama, LM Studio, llama.cpp, vLLM, vMLX ... I'm exhausted just typing these out, and that barely scratches the surface.

These generally vary in terms of "ease of use" vs "bare metal performance", but in my [limited] experience, and if you're just getting started (like I am), I would actually recommend using the easiest setup that works. For me, I tested at least 5 different options, and Ollama ended up being the simplest one that works.

Once you've gotten past runner-block - you now have the exhaustion and overload to pick a model. I didn't even know this when I started, but there are tons of open-source/open-weight models that you can download and run entirely free. Too many. Most online recommendations would likely steer you towards Gemma, Qwen or something similar. If you're getting started - don't go too crazy. The model I ultimately landed on as my main workhorse ended up being the one recommended on <https://modelfit.io/>. I'd probably recommend starting there.

Lastly - if you're doing agentic coding, you need a harness to interact with both your codebase and agent together. Again, tons of options here (too many!), but I had the most consistent success with Cline in VS Code. Could you do this other ways - absolutely. Pick what's comfortable for you.

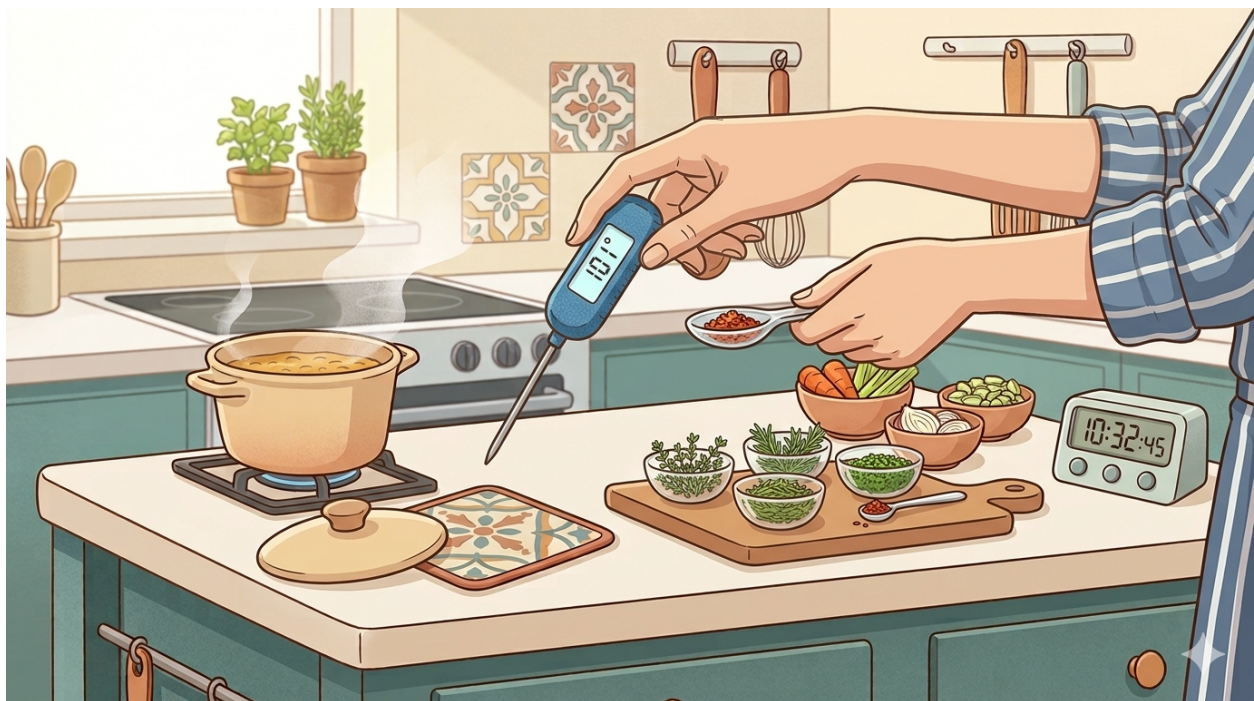
With the above guidance, I settled on my basic agent loop. I've attempted to fit different models in at times (and still occasionally do), but largely speaking, the bulk of my code comes from this set up.

There's a bit more to this section that I can share though - notably, the exact configuration of Cline and the agent guidelines. You will almost certainly need to adjust these from the defaults to get to a workable loop. Here's where I landed:

- Context Window - 120k
- Timeout - 300s

The above configuration made it much more likely that Ollama wouldn't time out on an initial response, nor churn beyond what was reasonable (most of the time). I probably could expand the window (and indeed I started larger), but I found that once the context window got consumed to around ~60-80k, my results got worse, even with Agentic context compaction turned on.

Lastly - don't go crazy with your agents guidelines files. Keep them brief, simple, to the point, and give the agent what it needs on every prompt and nothing more. Claude and other frontier models will write robust, detailed agents guidelines files, but in your own kitchen, size matters. Don't overcook it and you'll do just fine.



4. Know when (and be ready) to step in

Look - maybe Claude and others can vibe-code you a massive app without you knowing the first thing about coding, but local LLMs are not likely going to get you there without some solid know-how. I've lost count how many times I've had to step in to fix compiler errors and tests, tweak the UI, or adjust logic because prompting the LLM would've just been way slower. You should expect the mistakes and be ready to help your agent out as much as it's helping you.

Since you know this is a need, make your life easier if you can from the get go— in my iOS app, I should've set up command line scripts like Fastlane to compile and test from the beginning. It saves a ton of time to just fire off a command to verify the work of my agent. Yes, sometimes I let the agent run them itself, if I'm feeling bold, but that also uses precious context and often wastes a good amount of time where it'd actually be faster for me to do it myself.

Beyond that, know your domain. LLMs are likely going to output the code way faster than you can (even locally, accounting for the processing time), but if you don't know what you're working on, it's highly likely you're going to hit a point where you spin your wheels - a lot.

When you find yourself in a stuck-loop where the LLM just can't get a task done, always try and simplify. Break it down to smaller chunks, and ask the agent to slow down. Try having it just do 1 class or even 1 method if you really need to keep it small.



5. I say patience

Coding with a local LLM, no matter how good your hardware, is not faster than the big boys. It's still probably faster than your typing though. Be patient!

All this sounds like a lot of waiting around, and to be honest, it kind of is. While my local agent can plow through some great work, each task usually takes anywhere from around 5-10 minutes (or more) at minimum. That is not the same kind of instant feedback you're used to if you're vibing it with Claude or Cursor.

It's for this reason that having a good plan, a good set up, and a good idea of what good is going to look like is so important. When every task takes that long, you want to make sure you have the best chance of success every time.

Et - Voila!

So far, this has been one of the more fun coding assignments I've given myself in a while. It's simultaneously given me tons of learnings, while also scratching a major itch to reconnect with coding in a way that blasting through this with Claude would've missed out on. Just getting the agentic loop working on my computer forced a ton of creativity and research out of me, and I could not be happier with how things have turned out so far.

I'm still hard at work in the kitchen putting the final touches on my app, but I can't wait to share it with you all once it's fully baked.

If you've been interested in experimenting like I have, or just want to know more, reach out, I'd love to hear from you!

Happy hacking!